

NPTEL Course on
"Computer System Optimization for ML"

Lecture #41

NPTEL

Agenda

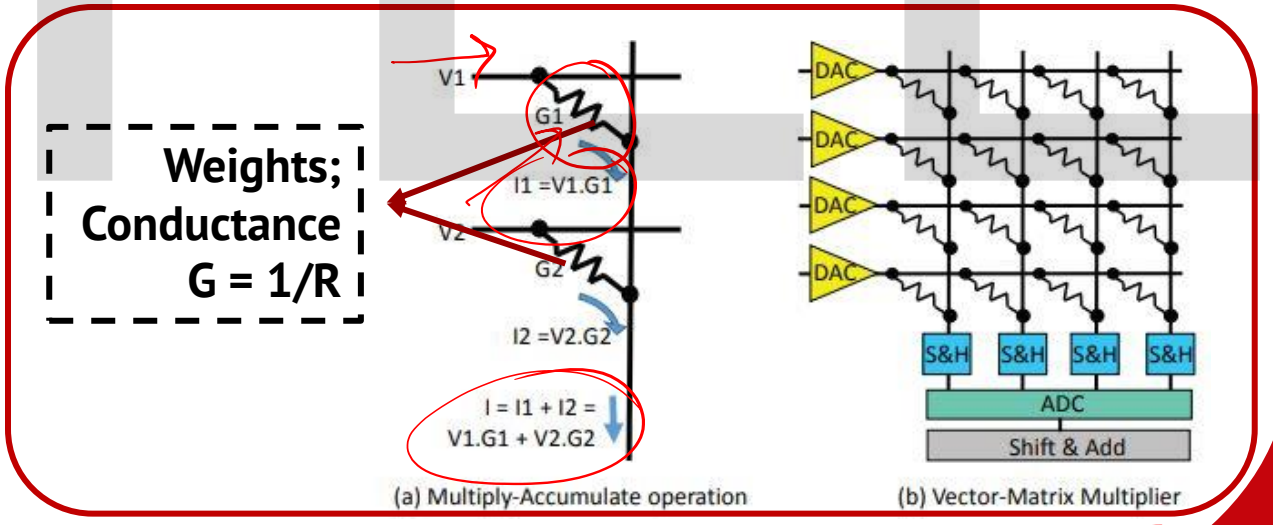
- Recap
- ISAAC
- Pipelayer

NPTEL

In-memory Computing (IMC) [1]

- **Analog computing: resistive (current) or capacitive (charge)**
- **Example: Resistive Random Access Memory (ReRAM or RRAM)**
  - A nonvolatile storage
  - Operates by changing the resistance of a special dielectric material
  - A family of Memristor
- **Advantage: No data movement between off chip memory**

[1] Shafiee, Ali, et al. "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars." ***ACM SIGARCH Computer Architecture News*** **44.3 (2016): 14-26.**



NPTEL

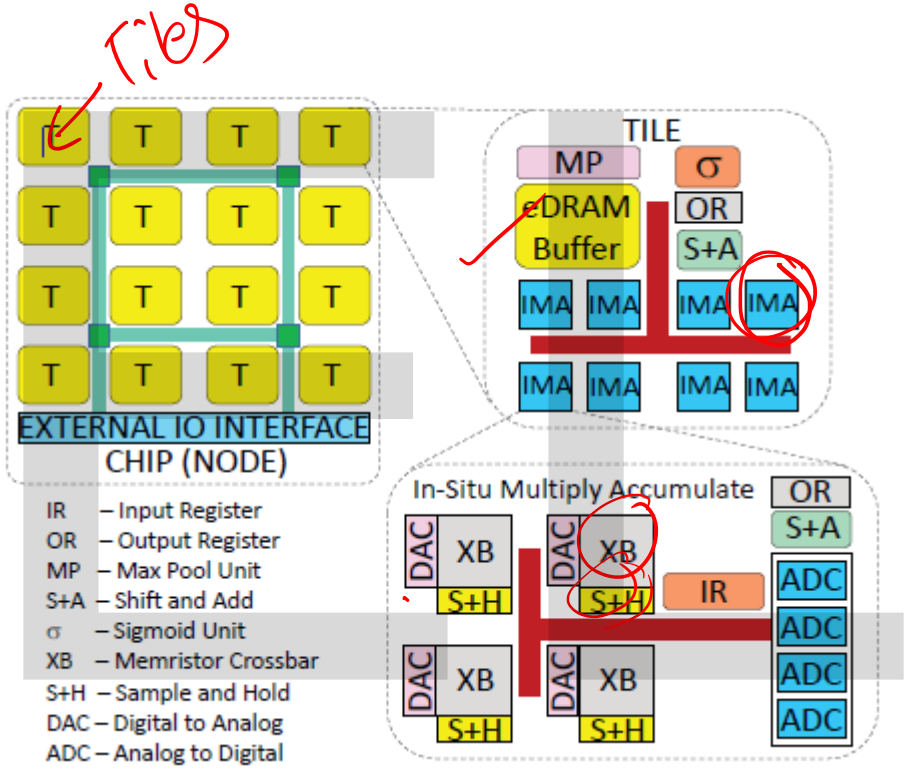
Agenda

- Recap
- **ISAAC**
- Pipelayer

NPTEL

Overview of ISAAC

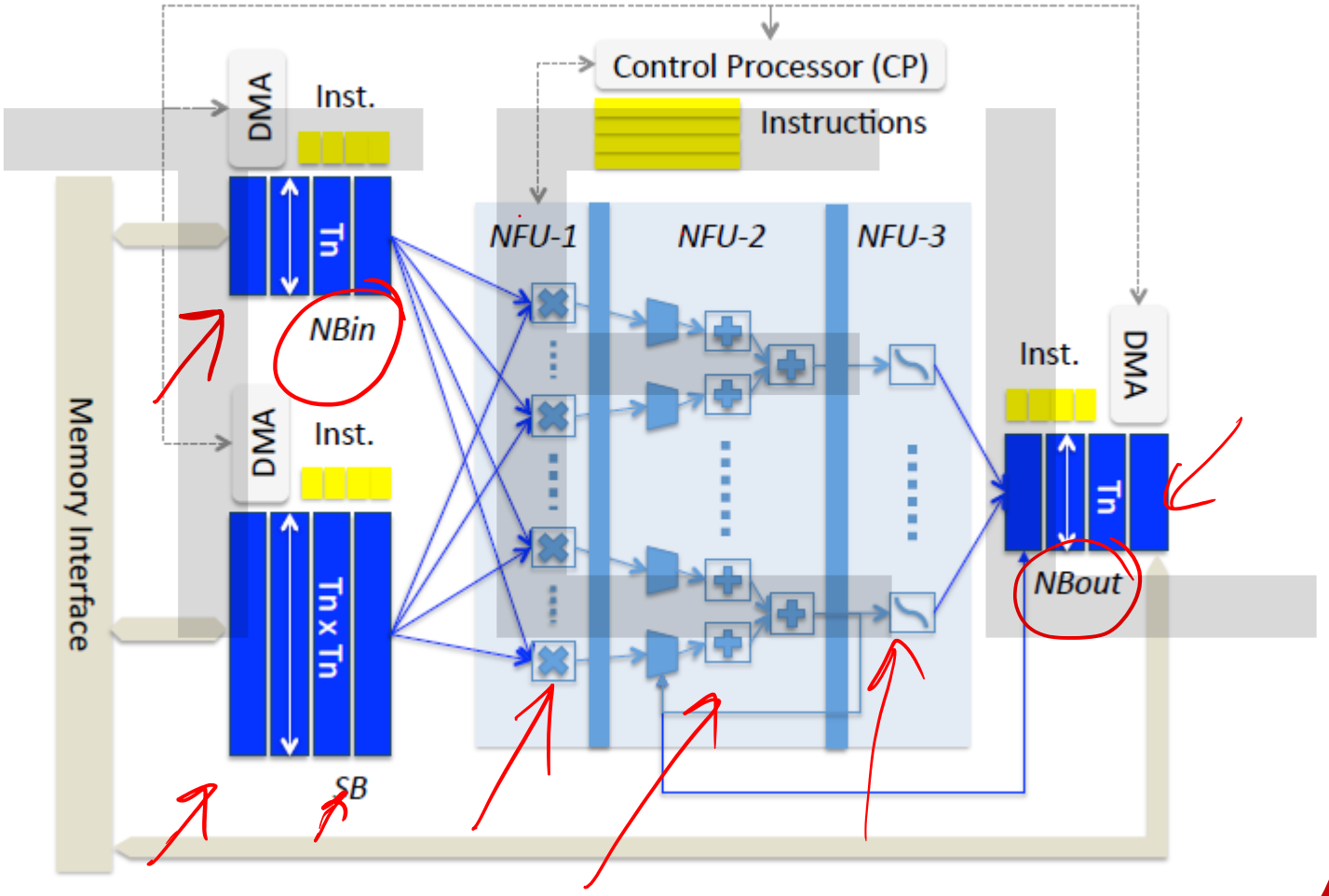
- **The ISAAC chip consists of number of tiles**
  - Connected with concentrated-mesh
  - eDRAM buffers to store input values
  - In-situ multiply-accumulate (IMA) units
  - Output registers to aggregate results
- **IMA**
  - Crossbars (XB)
  - DAC, ADC
  - Sample and hold
- **Only inference**



NPT

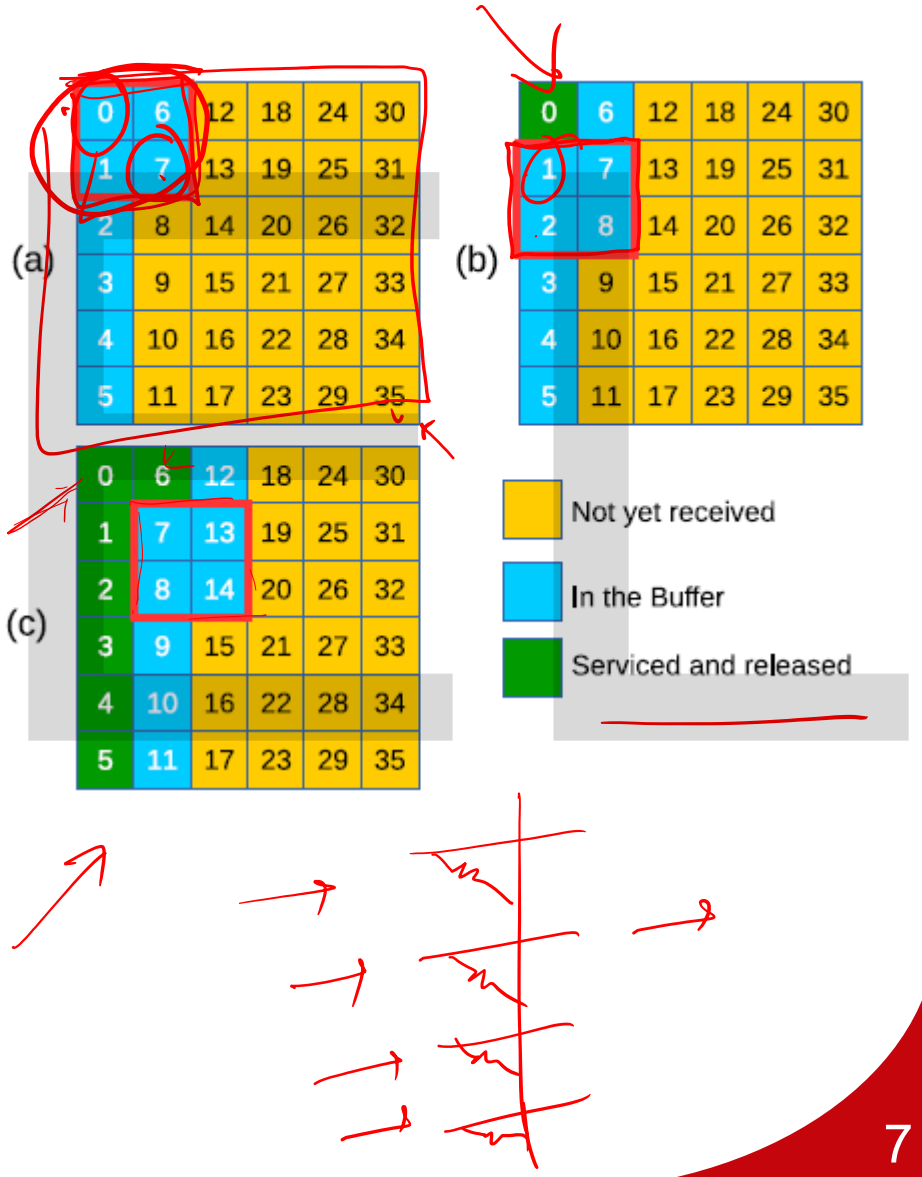
ISAAC Pipeline (1)

- **Revisiting DaDianNao**
- **Is there pipeline?**
  - No; it operates layer-by-layers
- **In IMC each memristor is responsible to store weights as well as compute**
  - High write energy makes it impossible to overwrite the weights



ISAAC Pipeline (2)

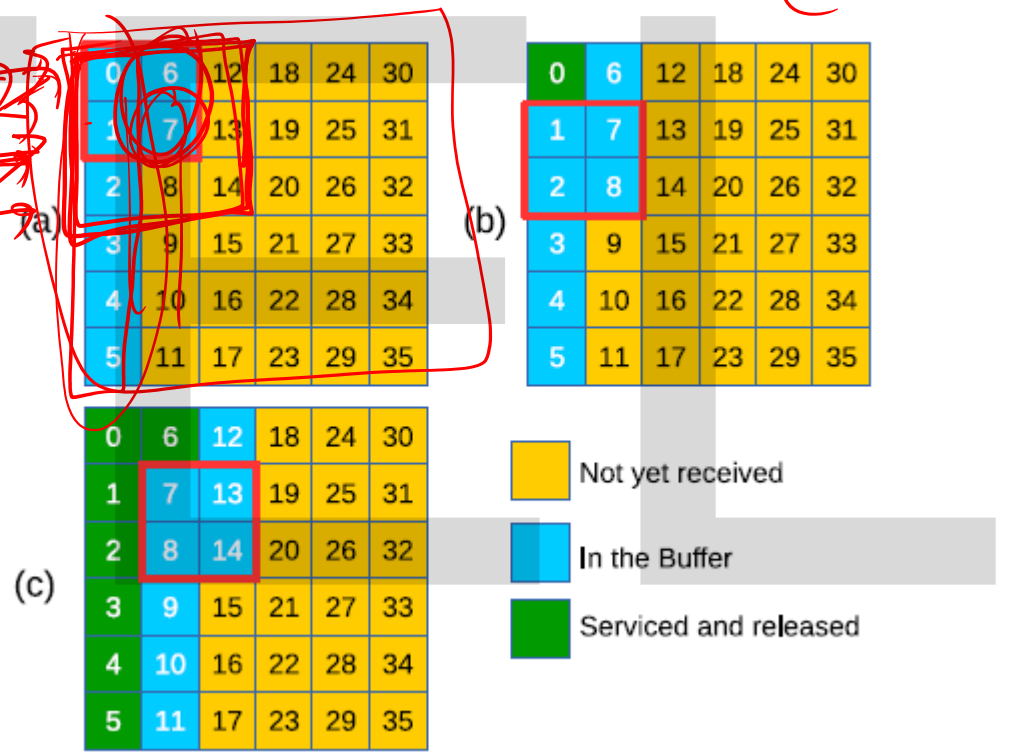
- **6x6 input feature mapped is being convolved with 2x2 kernel**
  - Assume a single IMA column has all 4 weights
- **Figure (a)**
  - Previous layer produces output 0-7
  - Placed in the input buffer of layer-I
  - 0,1,6,7 are fed to the IMA
  - One output (green box in Figure (b)) is produced
- **Figure (c) shows the state of the buffer after few steps**



NIDT

ISAAC Pipeline (3)

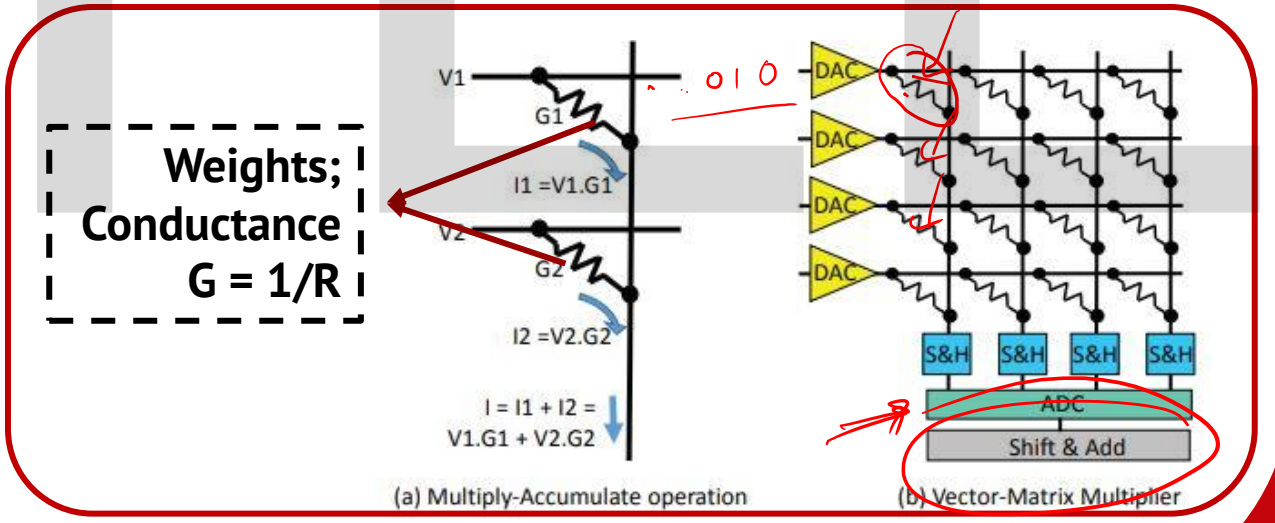
- **express eDRAM buffer requirement between two layers.**
  - Input feature map size: $N_x \times N_y$
  - Kernel size: $K_x \times K_y$
  - Number of input feature map: N_{if}
  - $(N_x \times (K_y - 1) + K_x) \times N_{if}$
- **what is the buffer requirement without pipelining?**
  - $N_x \times N_y \times N_{if}$



NPTH

Other Details

- **Inputs are quantized 16 bits**
- **What do we need ideally?**
  - 16-bit ADC
  - 2^{16} resistance level in each cell
  - Costly and error-prone
- **What is implemented in ISAAC?**
  - Inputs are 0/1 binary
  - 16 bits are supplied sequentially
  - Shifted and added



Results (comparison against DaDianNao)

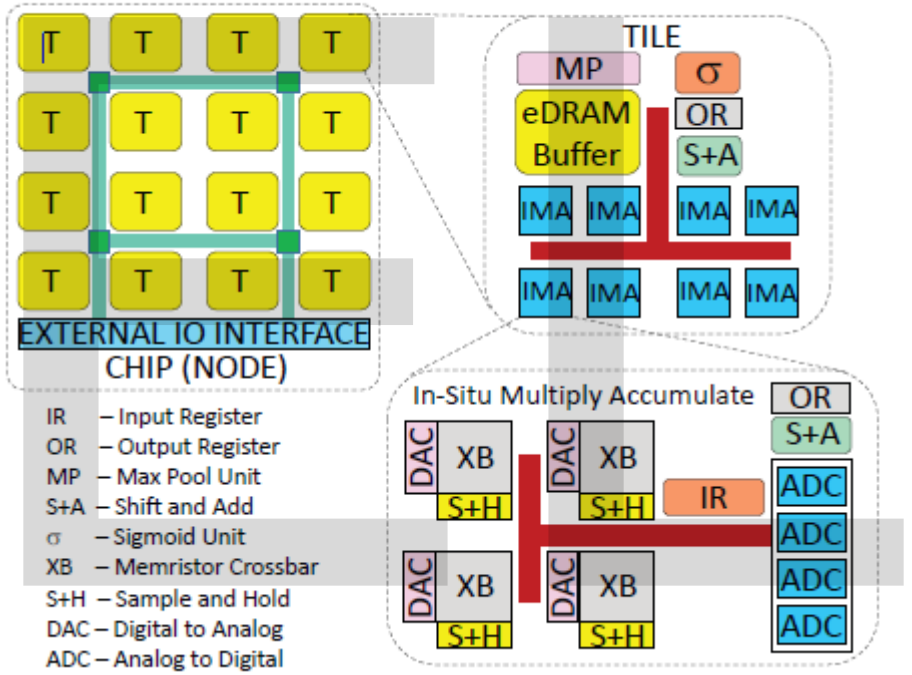
- Three key metrics
  - CE: Computational Efficiency (GOPS/(s x mm^2))
  - PE: Power Efficiency (GOPS/W)
  - SE: Storage Efficiency (MB/mm^2)

Architecture	CE $GOPs/(s \times mm^2)$	PE $GOPs/W$	SE MB/mm^2
DaDianNao	63.46	286.4	0.41
ISAAC-CE	478.95	363.7	0.74
ISAAC-PE	466.8	380.7	0.71
ISAAC-SE	140.3	255.3	54.8

NPTEL

Overview of ISAAC

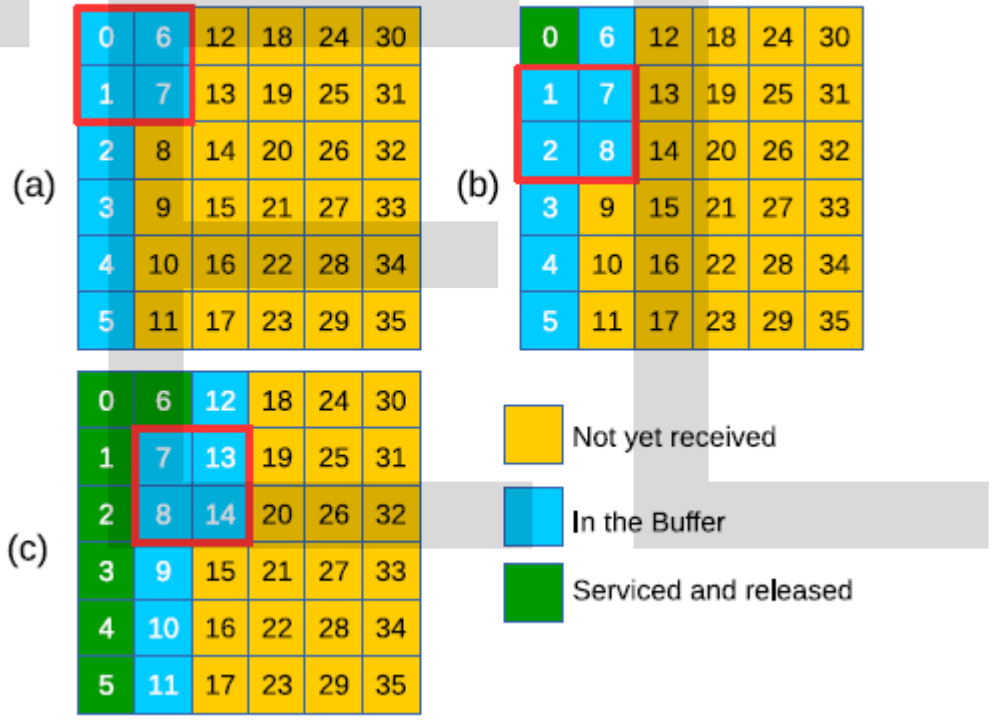
- **The ISAAC chip consists of number of tiles**
  - Connected with concentrated-mesh
  - eDRAM buffers to store input values
  - In-situ multiply-accumulate (IMA) units
  - Output registers to aggregate results
- **IMA**
  - Crossbars (XB)
  - DAC, ADC
  - Sample and hold
- **Only inference**



NPT

ISAAC Pipeline (3)

- **express eDRAM buffer requirement between two layers.**
  - Input feature map size: $N_x \times N_y$
  - Kernel size: $K_x \times K_y$
  - Number of input feature map: N_{if}
  - $(N_x \times (K_y - 1) + K_x) \times N_{if}$
- **what is the buffer requirement without pipelining?**
  - $N_x \times N_y \times N_{if}$



NDT

Agenda

- Recap
- ISAAC
- **Pipelayer**

NPTEL

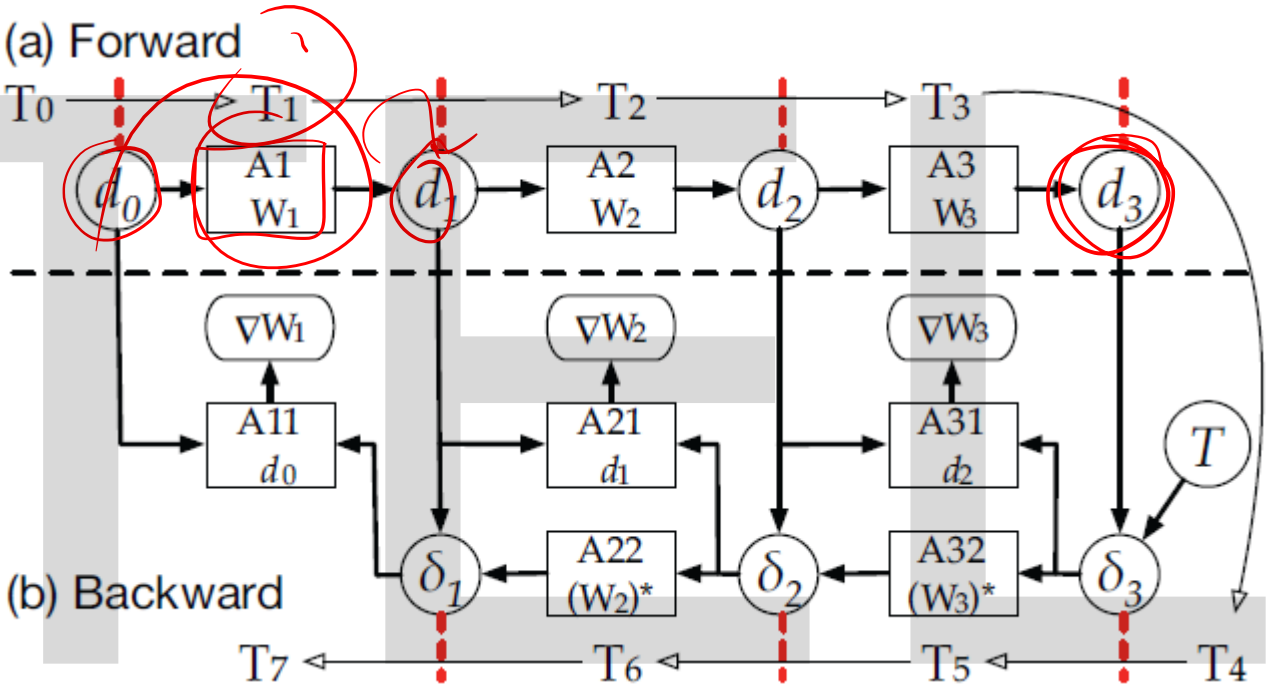
PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning

- **PipeLayer considers both training and inference**
  - More complex than inference
  - Weight update to a layer depend on the previous layer's error and the data of the earlier forward computation
- **Spike-based scheme to eliminate the overhead of DACs and ADCs**
- **Main idea**
  - Segregated into morphable subarrays and memory subarrays
  - Morphable subarrays are used to store both compute and storage
  - Memory subarrays are used only for storage

NPTEL

Training in PipeLayer (1)

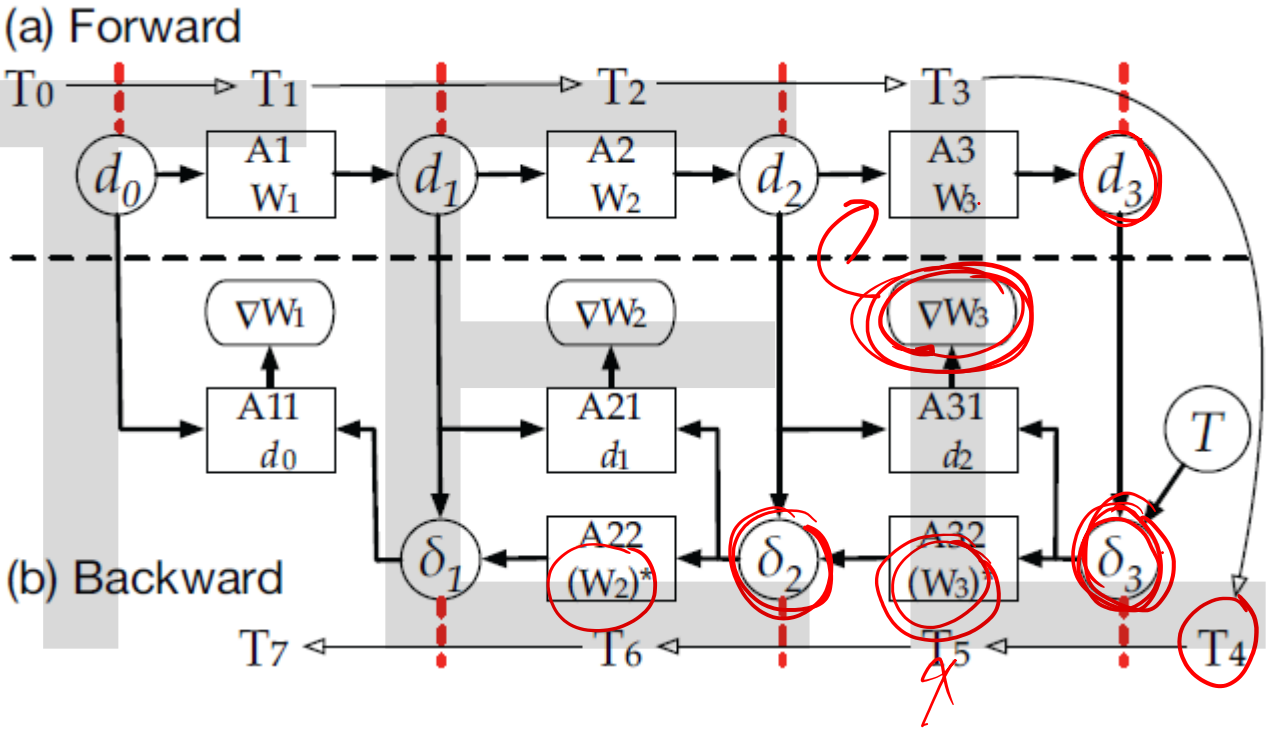
- **Rectangles denote morphable subarrays; circles denote memory subarrays**
- **Forward propagation**
  - Input d_0 enters A1 (morphable subarray) at T1
  - Performs matrix-vector-multiplication
  - Outputs of each layer are stored in d_0
  - Results of the forward propagation are stored at d_3



NPT

Training in PipeLayer (2)

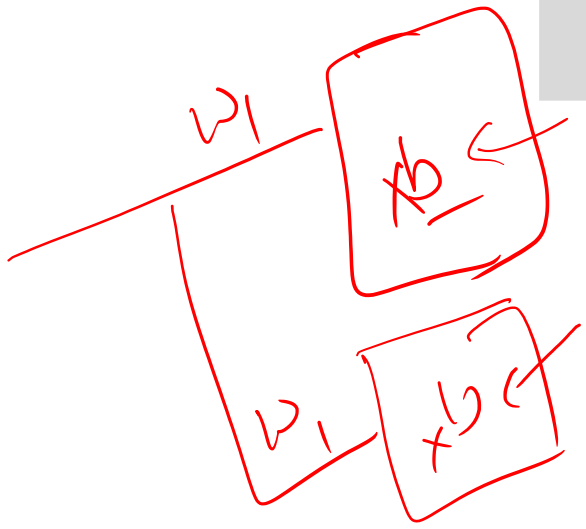
- **Backward propagation**
  - Generates error at each layer-l (δ_l)
  - Generates partial derivatives (ΔW_l)
  - δ_3 is computed at T_4
  - Stored in memory subarrays
  - In T_5
    - ΔW_3 is computed
    - δ_2 is computed
  - In case of batch size more than 1, the weight gradients are stored in buffers

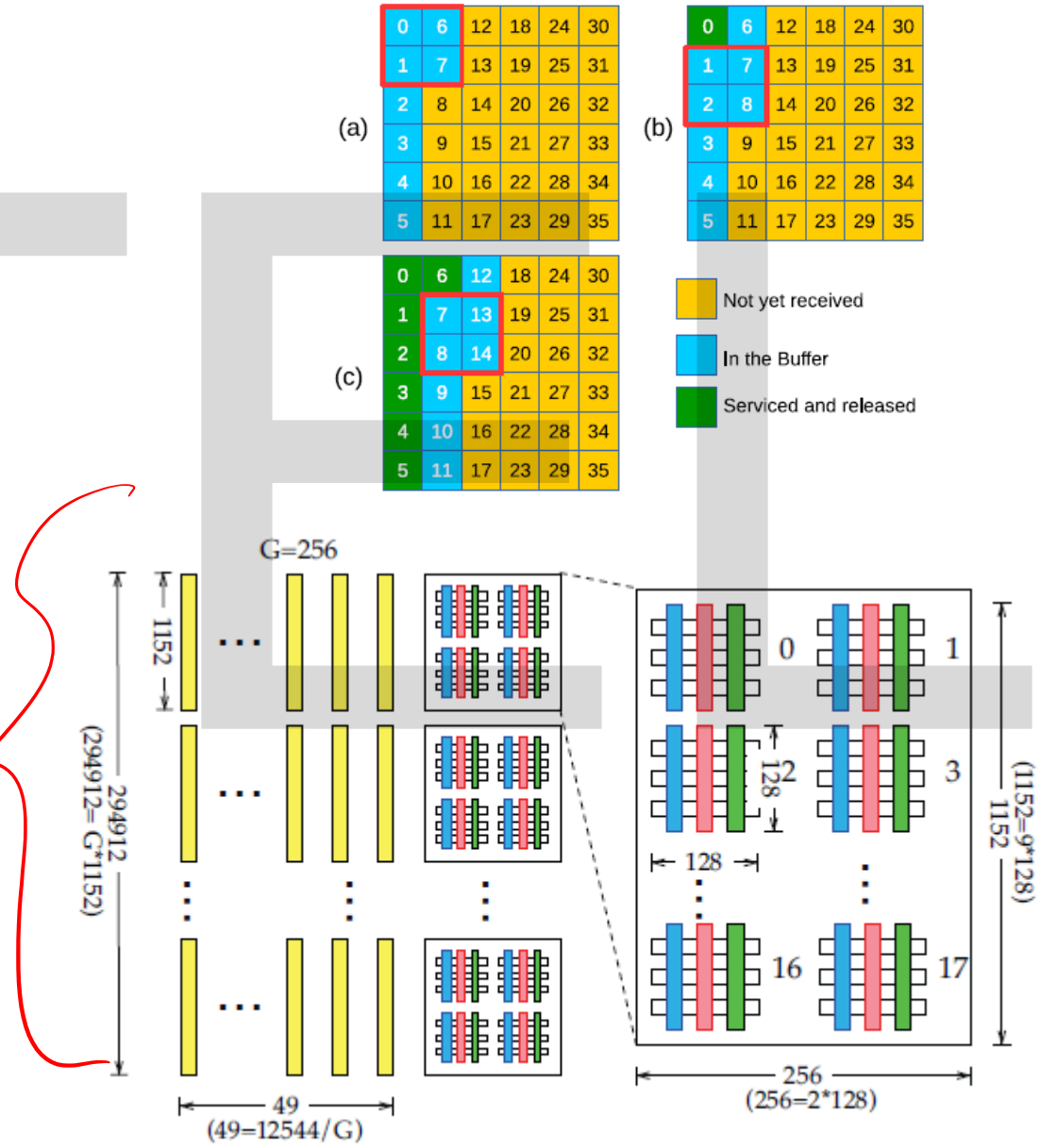


NPT

Training in PipeLayer (3)

- **ISAAC can results in pipeline stall**
  - Why?
- **Intra Layer Parallelism**
  - Parallelism granularity (G)
  - Duplication of weights
  - Trade-off between hardware resource and performance

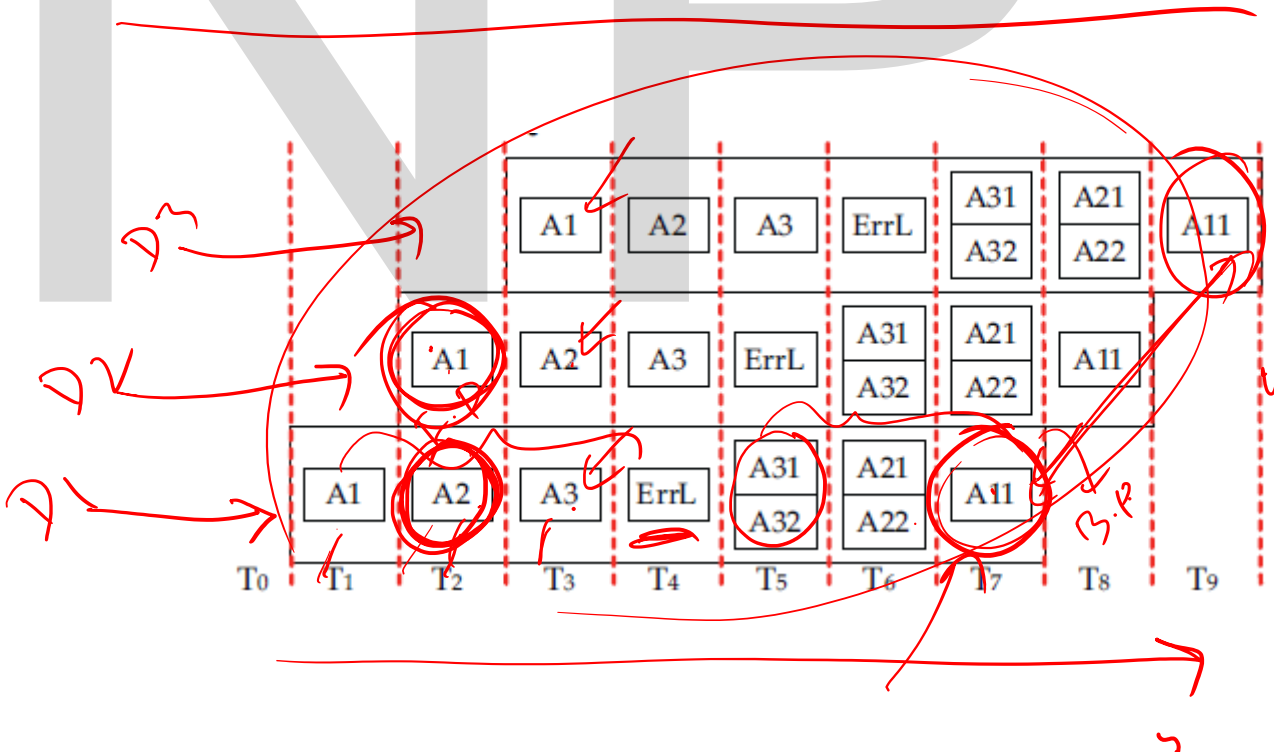


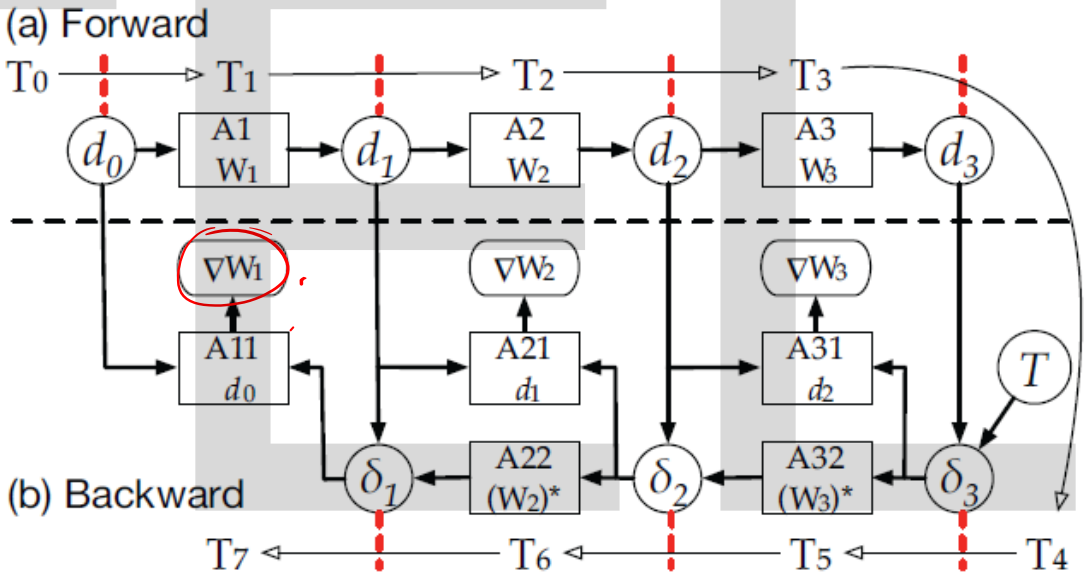


NPTEL

Training in PipeLayer (4)

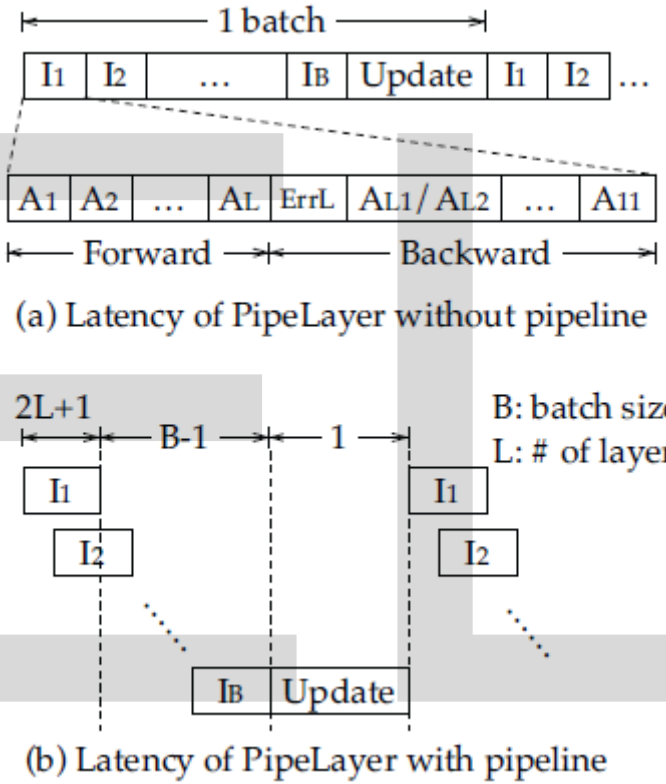
- A new input can enter the pipeline every cycle within a batch
- At the end of a batch, a new input belonging to the next batch cannot enter the pipeline until all inputs in the previous batch are processed





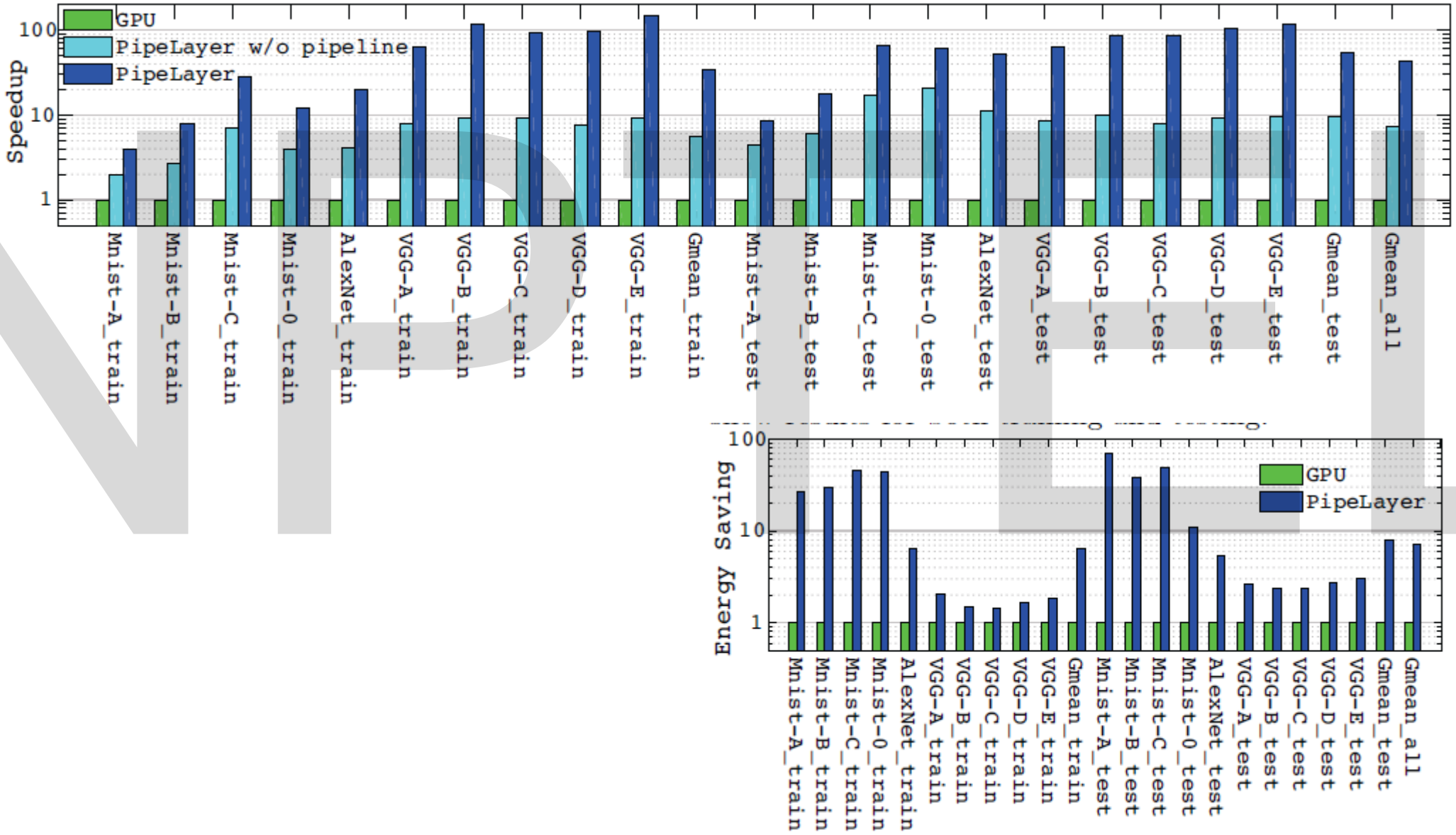
Non-pipelined vs Pipelined: Comparison

- **Assume the DNN has L layers and each layer gets executed at each (logical) cycles**
  - Forward computation: L cycles
  - Backward computation: (L+1) cycles
- **For the whole batch**
  - (2L+1)B + 1 cycles
- **For N inputs**
  - (2L+1)N + N/B cycles
- **In pipelined**
  - (N/B)(2L+B+1)



NPTEL

Results


GPU
PipeLayer w/o pipeline
PipeLayer
Speedup
100
10
1
Mnist-A_train
Mnist-B_train
Mnist-C_train
Mnist-0_train
AlexNet_train
VGG-A_train
VGG-B_train
VGG-C_train
VGG-D_train
VGG-E_train
Gmean_train
Mnist-A_test
Mnist-B_test
Mnist-C_test
Mnist-0_test
AlexNet_test
VGG-A_test
VGG-B_test
VGG-C_test
VGG-D_test
VGG-E_test
Gmean_test
Gmean_all
Energy Saving
GPU
PipeLayer

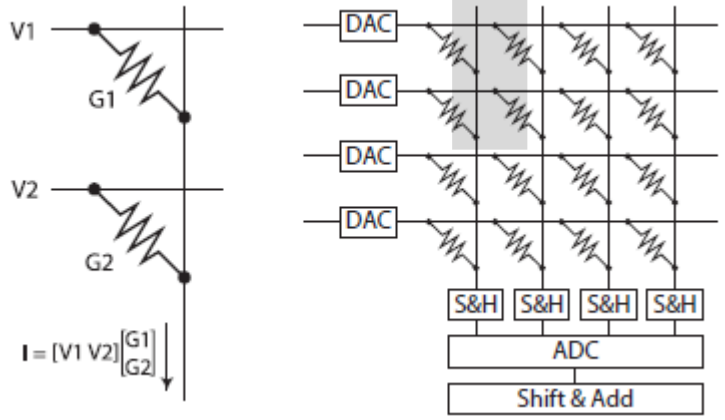
Agenda

- Recap
- Pipelayer
- **Atomlayer**

NPTEL

AtomLayer: A Universal ReRAM-Based CNN Accelerator with Atomic Layer Computation

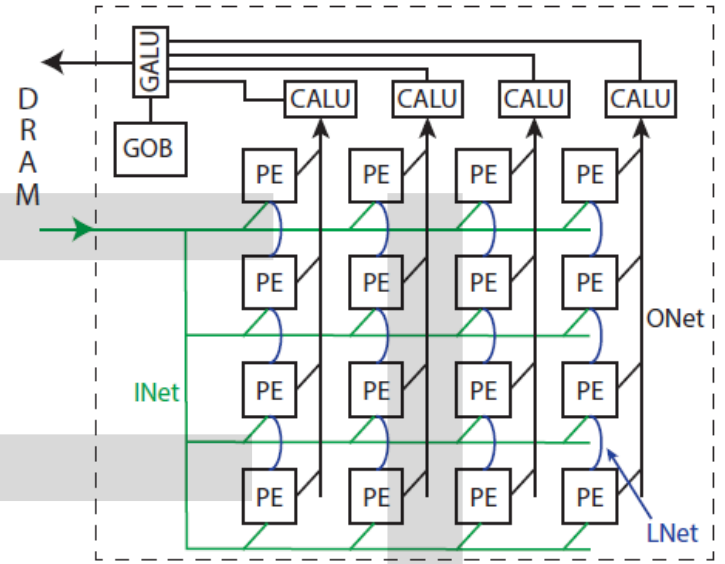
- Atomic layer computation
  - Only one layer at a time (going back to non-pipeline ☺)
  - Eliminates long latency, pipeline bubbles, large on-chip storage requirement
- Rotating crossbars structure
- Row-disjoint filter mapping and a data reuse system

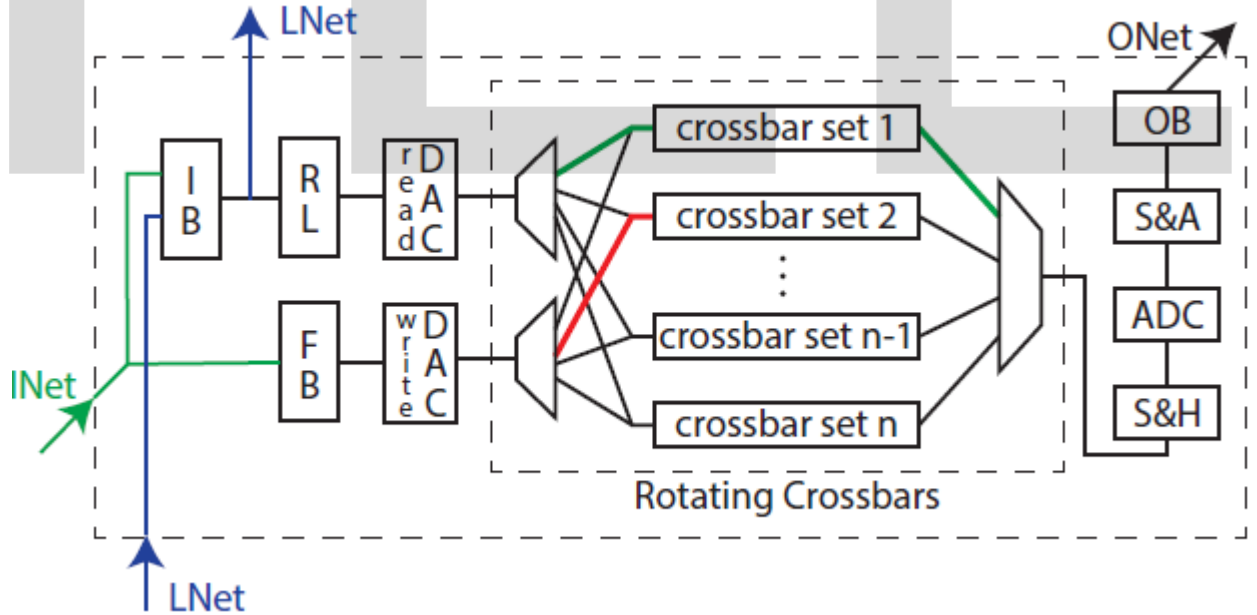


NPTEL

AtomLayer Architecture

- **Processing elements (PE), on-chip network, ALU tree, global output buffer**
- **ALU tree**
  - Column ALU (CALU)
  - Global ALU (GALU)
- **PE**
  - Rotating crossbars
  - Peripheral devices
  - Four buffers

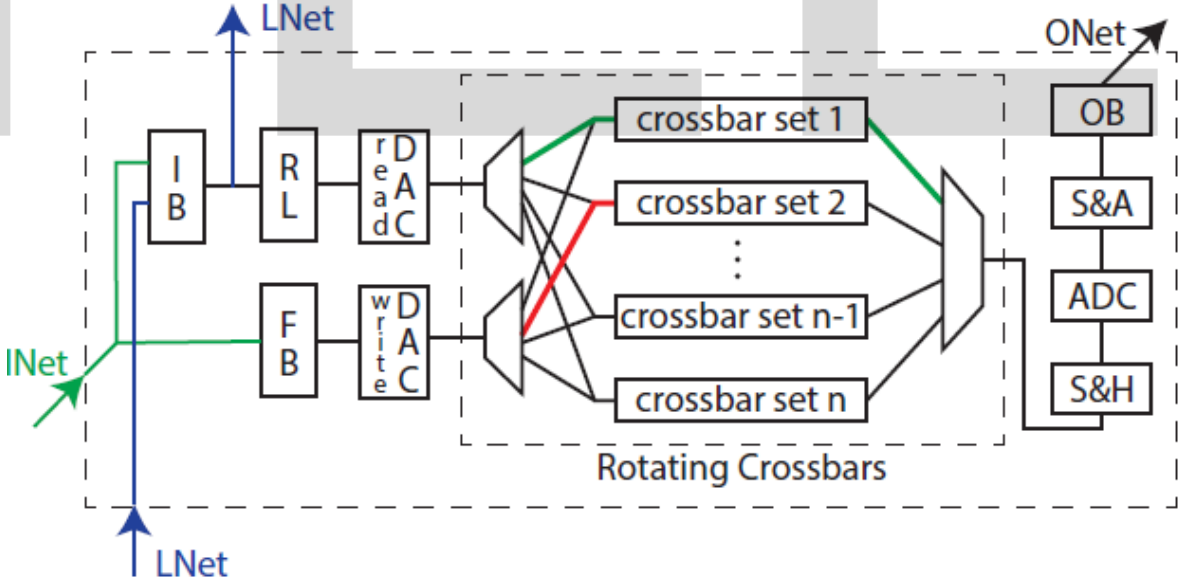




NPTEL

Rotating Crossbar in AtomLayer

- **Key component to perform atomic layer computation**
- **Crossbars are divided into crossbar set**
  - Number of ReRAM to match the performance of the peripheral devices
- **In every cycle, once crossbar set is connected to read DACs and S&H for computation**
- **Number of sets is no less than number of NN layers**



Results

	GOPs/s/mm^2	GOPs/W	mm^2	W
160PE (w/o FC)	615.7	682.5	5.32	4.80
160PE (w/ FC)	475.6	682.5	6.89	4.80
ISAAC	478.9	627.5	85.4	65.8
PipeLayer	1485	142.9	82.6	168.6

	VGG-19 (ms)	ResNet-152 (ms)
160PE	6.92	4.01
ISAAC	8.00	43.46
PipeLayer	2.60	14.13

NPTEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)